STUDENT MANAGEMENT SYSTEM

Complete Code Explanation — Line by Line

A beginner-friendly guide to Python, JSON file handling, validation, CRUD operations, statistics, sorting, and automated tests.

Prepared from the exact code supplied in the request.

1. Project overview

This command-line application stores student records in a JSON file named `students.json`. Each record contains an ID, name, age, course, and marks. The menu supports adding, viewing, searching, updating, deleting, calculating statistics, sorting records, running tests, and exiting.

Main concepts used

• Variables and dictionaries hold each student's details.

• Lists hold multiple student dictionaries.

• Functions divide the program into reusable tasks.

• if/elif/else statements implement decisions such as validation and grades.

• while and for loops repeat input prompts and process records.

• try/except blocks handle file and conversion errors.

• json.load/json.dump load and save persistent records.

• Assertions test important rules automatically.

Important behavior to understand

The pass threshold is 35 marks. The grade boundaries are A: 90–100, B: 80–89.999..., C: 70–79.999..., D: 60–69.999..., E: 35–59.999..., and F: below 35. A score of 85 or more receives the extra “Congratulations” message, even though the letter grade remains B until 90.

The file path is based on `os.getcwd()`, which means the JSON file is stored in the current working directory from which Python is launched—not necessarily beside the script.

2. Line-by-line explanation

Every non-empty source line is listed below with an explanation. Blank lines are also identified because they separate code sections. For multi-line expressions, continuation lines are explained in the context of the expression they belong to.

```
L001 import json
```

Loads Python's JSON tools to read and write student records.

```
L002 import os
```

Loads operating-system path utilities used to locate students.json.

```
L003 import math
```

Loads mathematical helpers, including checks for finite numeric marks.

```
L004 [blank line]
```

Blank line used to separate logical sections and improve readability.

```
L005 # File path
```

Comment for the reader; Python ignores this line during execution.

```
L006 FILE_NAME = os.path.join(os.getcwd(), "students.json")
```

Builds the path to students.json in the current working directory.

```
L007 [blank line]
```

Blank line used to separate logical sections and improve readability.

```
L008 [blank line]
```

Blank line used to separate logical sections and improve readability.

```
L009 # ==================== FILE HANDLING ====================
```

Section heading comment: marks the beginning of a related group of functions; it does not execute.

```
L010 [blank line]
```

Blank line used to separate logical sections and improve readability.

Function: load_students | Source lines around 11

```
L011 def load_students():
```

Defines the function that loads and validates saved student records.

```
L012     try:
```

Starts an exception-handling block for an operation that may fail.

```
L013         with open(FILE_NAME, "r", encoding="utf-8") as file:
```

Opens the JSON file for reading; the context manager closes it automatically.

```
L014             data = json.load(file)
```

Parses the JSON file and converts its contents into Python objects.

```
L015 [blank line]
```

Blank line used to separate logical sections and improve readability.

```
L016         if not isinstance(data, list):
```

Checks a condition and runs the indented block only when it is true.

```
L017             print("Invalid JSON format. Starting with empty records.")
```

Displays a message, result, warning, or formatted output to the terminal.

```
L018             return []
```

Returns an empty list, meaning no usable student records are available.

```
L019 [blank line]
```

Blank line used to separate logical sections and improve readability.

```
L020         students = []
```

Creates an empty list to collect valid student records.

```
L021 [blank line]
```

Blank line used to separate logical sections and improve readability.

```
L022         for student in data:
```

Starts a loop that processes each item in the specified iterable.

```
L023             if not isinstance(student, dict):
```
Checks a condition and runs the indented block only when it is true.
```
L024                 continue
```
Skips the rest of this loop iteration and proceeds to the next item/input attempt.
```
L025 [blank line]
```
Blank line used to separate logical sections and improve readability.
```
L026             if not all(
```
Checks a condition and runs the indented block only when it is true.
```
L027                 key in student
```
Checks whether a required field exists in the student dictionary.
```
L028                 for key in ("id", "name", "age", "course", "marks")
```
Starts a loop that processes each item in the specified iterable.
```
L029             ):
```
Closes the dictionary, call, list, or grouped expression opened above.
```
L030                 continue
```
Skips the rest of this loop iteration and proceeds to the next item/input attempt.
```
L031 [blank line]
```
Blank line used to separate logical sections and improve readability.
```
L032             if not all(
```
Checks a condition and runs the indented block only when it is true.
```
L033                 isinstance(student[key], str)
```
Checks the object's type before the program relies on it.
```
L034                 for key in ("id", "name", "course")
```
Starts a loop that processes each item in the specified iterable.
```
L035             ):
```
Closes the dictionary, call, list, or grouped expression opened above.
```
L036                 continue
```
Skips the rest of this loop iteration and proceeds to the next item/input attempt.
```
L037 [blank line]
```
Blank line used to separate logical sections and improve readability.
```
L038             if (
```
Checks a condition and runs the indented block only when it is true.
```
L039                 not student["id"]
```
Combines or negates Boolean conditions used for validation.
```
L040                 or not student["id"].isascii()
```
Combines or negates Boolean conditions used for validation.
```
L041                 or not student["id"].isalnum()
```
Combines or negates Boolean conditions used for validation.
```
L042                 or not student["name"]
```
Combines or negates Boolean conditions used for validation.
```
L043                 or not is_valid_name(student["name"])
```
Combines or negates Boolean conditions used for validation.
```
L044                 or not student["course"].strip()
```
Combines or negates Boolean conditions used for validation.
```
L045                 or not any(c.isalpha() for c in student["course"])
```
Combines or negates Boolean conditions used for validation.
```
L046             ):
```
Closes the dictionary, call, list, or grouped expression opened above.
```
L047                 continue
```

Skips the rest of this loop iteration and proceeds to the next item/input attempt.

```
L048 [blank line]
```

Blank line used to separate logical sections and improve readability.

```
L049             age = student["age"]
```

Reads the student's age field from the dictionary.

```
L050             marks = student["marks"]
```

Reads the student's marks field from the dictionary.

```
L051 [blank line]
```

Blank line used to separate logical sections and improve readability.

```
L052             if (
```

Checks a condition and runs the indented block only when it is true.

```
L053                 isinstance(age, bool)
```

Checks the object's type before the program relies on it.

```
L054                 or not isinstance(age, int)
```

Combines or negates Boolean conditions used for validation.

```
L055                 or not 1 <= age <= 120
```

Assigns the expression on the right to the variable or dictionary field on the left.

```
L056             ):
```

Closes the dictionary, call, list, or grouped expression opened above.

```
L057                 continue
```

Skips the rest of this loop iteration and proceeds to the next item/input attempt.

```
L058 [blank line]
```

Blank line used to separate logical sections and improve readability.

```
L059             if (
```

Checks a condition and runs the indented block only when it is true.

```
L060                 isinstance(marks, bool)
```

Checks the object's type before the program relies on it.

```
L061                 or not isinstance(marks, (int, float))
```

Combines or negates Boolean conditions used for validation.

```
L062                 or not math.isfinite(marks)
```

Combines or negates Boolean conditions used for validation.

```
L063                 or not 0 <= marks <= 100
```

Assigns the expression on the right to the variable or dictionary field on the left.

```
L064             ):
```

Closes the dictionary, call, list, or grouped expression opened above.

```
L065                 continue
```

Skips the rest of this loop iteration and proceeds to the next item/input attempt.

```
L066 [blank line]
```

Blank line used to separate logical sections and improve readability.

```
L067             # Avoid duplicate IDs in existing JSON data
```

Comment for the reader; Python ignores this line during execution.

```
L068             if find_student(students, student["id"]):
```

Checks a condition and runs the indented block only when it is true.

```
L069                 continue
```

Skips the rest of this loop iteration and proceeds to the next item/input attempt.

```
L070 [blank line]
```

Blank line used to separate logical sections and improve readability.

```
L071             students.append(student)
```

Adds the student dictionary to the in-memory list.

```
L072 [blank line]
```

Blank line used to separate logical sections and improve readability.

```
L073         return students
```

Returns the calculated value or selected record to the caller.

```
L074 [blank line]
```

Blank line used to separate logical sections and improve readability.

```
L075     except FileNotFoundError:
```

Handles this specific error so the program can respond gracefully instead of crashing.

```
L076         return []
```

Returns an empty list, meaning no usable student records are available.

```
L077     except json.JSONDecodeError:
```

Handles this specific error so the program can respond gracefully instead of crashing.

```
L078         print("Warning: students.json contains invalid JSON.")
```

Displays a message, result, warning, or formatted output to the terminal.

```
L079         return []
```

Returns an empty list, meaning no usable student records are available.

```
L080     except OSError as error:
```

Handles this specific error so the program can respond gracefully instead of crashing.

```
L081         print(f"Error reading records: {error}")
```

Displays a message, result, warning, or formatted output to the terminal.

```
L082         return []
```

Returns an empty list, meaning no usable student records are available.

```
L083 [blank line]
```

Blank line used to separate logical sections and improve readability.

```
L084 [blank line]
```

Blank line used to separate logical sections and improve readability.

Function: save_students | Source lines around 85

```
L085 def save_students(students):
```

Defines the function that writes student records to the JSON file.

```
L086     try:
```

Starts an exception-handling block for an operation that may fail.

```
L087         with open(FILE_NAME, "w", encoding="utf-8") as file:
```

Opens the JSON file for writing; existing contents are replaced, and the context manager closes it.

```
L088             json.dump(
```

Serializes the student list to JSON and writes it to the opened file.

```
L089                 students,
```

Part of the current expression: it supplies an argument, condition, value, or continuation to the statement above.

```
L090                 file,
```

Part of the current expression: it supplies an argument, condition, value, or continuation to the statement above.

```
L091                 indent=4,
```

Formats the JSON output with indentation for readability.

```
L092                 ensure_ascii=False,
```

Allows Unicode characters to be written directly instead of escaped.

```
L093                 allow_nan=False
```

Prevents non-standard NaN or infinity values from being written to JSON.

```
L094             )
```

Closes the dictionary, call, list, or grouped expression opened above.

```
L095         return True
```

Reports that the operation succeeded.

```
L096 [blank line]
```

Blank line used to separate logical sections and improve readability.

```
L097     except (OSError, ValueError) as error:
```

Handles this specific error so the program can respond gracefully instead of crashing.

```
L098         print(f"Error saving records: {error}")
```

Displays a message, result, warning, or formatted output to the terminal.

```
L099         return False
```

Reports that the operation failed.

```
L100 [blank line]
```

Blank line used to separate logical sections and improve readability.

```
L101 [blank line]
```

Blank line used to separate logical sections and improve readability.

```
L102 # ==================== INPUT VALIDATION ====================
```

Section heading comment: marks the beginning of a related group of functions; it does not execute.

```
L103 [blank line]
```

Blank line used to separate logical sections and improve readability.

Function: get_non_empty_text | Source lines around 104

```
L104 def get_non_empty_text(prompt, field_name):
```

Defines a reusable input helper that rejects blank text.

```
L105     while True:
```

Repeats the block while the condition remains true.

```
L106         value = input(prompt).strip()
```

Reads text typed by the user; input() always initially returns a string.

```
L107 [blank line]
```

Blank line used to separate logical sections and improve readability.

```
L108         if value:
```

Checks a condition and runs the indented block only when it is true.

```
L109             return value
```

Returns the calculated value or selected record to the caller.

```
L110 [blank line]
```

Blank line used to separate logical sections and improve readability.

```
L111         print(f"{field_name} cannot be empty.")
```

Displays a message, result, warning, or formatted output to the terminal.

```
L112 [blank line]
```

Blank line used to separate logical sections and improve readability.

```
L113 [blank line]
```

Blank line used to separate logical sections and improve readability.

Function: is_valid_name | Source lines around 114

```
L114 def is_valid_name(name):
```

Defines the rules used to decide whether a name is acceptable.

```
L115     return (
```

Returns the calculated value or selected record to the caller.

```
L116         bool(name)
```

Part of the current expression: it supplies an argument, condition, value, or continuation to the statement above.

```
L117         and any(char.isalpha() for char in name)
```

Combines or negates Boolean conditions used for validation.

```
L118         and all(
```

Combines or negates Boolean conditions used for validation.

```
L119             char.isalpha() or char in " -'."
```

Part of the current expression: it supplies an argument, condition, value, or continuation to the statement above.

```
L120             for char in name
```

Starts a loop that processes each item in the specified iterable.

```
L121         )
```

Closes the dictionary, call, list, or grouped expression opened above.

```
L122     )
```

Closes the dictionary, call, list, or grouped expression opened above.

```
L123 [blank line]
```

Blank line used to separate logical sections and improve readability.

```
L124 [blank line]
```

Blank line used to separate logical sections and improve readability.

Function: get_student_name | Source lines around 125

```
L125 def get_student_name(prompt):
```

Defines a loop that keeps asking until the name passes validation.

```
L126     while True:
```

Repeats the block while the condition remains true.

```
L127         name = input(prompt).strip()
```

Reads text typed by the user; input() always initially returns a string.

```
L128 [blank line]
```

Blank line used to separate logical sections and improve readability.

```
L129         if is_valid_name(name):
```

Checks a condition and runs the indented block only when it is true.

```
L130             return name
```

Returns the calculated value or selected record to the caller.

```
L131 [blank line]
```

Blank line used to separate logical sections and improve readability.

```
L132         print(
```

Displays a message, result, warning, or formatted output to the terminal.

```
L133             "Invalid name! Use letters, spaces, hyphens, "
```

Provides a string literal used as data, a message, or a dictionary key.

```
L134             "apostrophes or periods only."
```

Provides a string literal used as data, a message, or a dictionary key.

```
L135         )
```

Closes the dictionary, call, list, or grouped expression opened above.

```
L136 [blank line]
```

Blank line used to separate logical sections and improve readability.

```
L137 [blank line]
```

Blank line used to separate logical sections and improve readability.

Function: get_student_id | Source lines around 138

```
L138 def get_student_id(prompt):
```

Defines a loop that accepts only non-empty ASCII letters and digits for IDs.

```
L139     while True:
```

Repeats the block while the condition remains true.

```
L140         student_id = input(prompt).strip()
```

Reads text typed by the user; input() always initially returns a string.

```
L141 [blank line]
```

Blank line used to separate logical sections and improve readability.

```
L142         if student_id and student_id.isascii() and student_id.isalnum():
```

Checks a condition and runs the indented block only when it is true.

```
L143             return student_id
```

Returns the calculated value or selected record to the caller.

```
L144 [blank line]
```

Blank line used to separate logical sections and improve readability.

```
L145         print(
```

Displays a message, result, warning, or formatted output to the terminal.

```
L146             "Invalid Student ID! Use English letters and numbers only. "
```

Provides a string literal used as data, a message, or a dictionary key.

```
L147             "Example: ST101"
```

Provides a string literal used as data, a message, or a dictionary key.

```
L148         )
```

Closes the dictionary, call, list, or grouped expression opened above.

```
L149 [blank line]
```

Blank line used to separate logical sections and improve readability.

```
L150 [blank line]
```

Blank line used to separate logical sections and improve readability.

Function: get_course | Source lines around 151

```
L151 def get_course(prompt):
```

Defines a loop that accepts a course name containing at least one letter.

```
L152     while True:
```

Repeats the block while the condition remains true.

```
L153         course = input(prompt).strip()
```

Reads text typed by the user; input() always initially returns a string.

```
L154 [blank line]
```

Blank line used to separate logical sections and improve readability.

```
L155         if course and any(char.isalpha() for char in course):
```

Checks a condition and runs the indented block only when it is true.

```
L156             return course
```

Returns the calculated value or selected record to the caller.

```
L157 [blank line]
```

Blank line used to separate logical sections and improve readability.

```
L158         print("Invalid course! Enter a course containing letters.")
```

Displays a message, result, warning, or formatted output to the terminal.

```
L159 [blank line]
```

Blank line used to separate logical sections and improve readability.

```
L160 [blank line]
```

Blank line used to separate logical sections and improve readability.

Function: get_integer | Source lines around 161

```
L161 def get_integer(prompt, minimum, maximum):
```

Defines a helper for integer input within an inclusive minimum/maximum range.

```
L162     while True:
```

Repeats the block while the condition remains true.

```
L163         value = input(prompt).strip()
```

Reads text typed by the user; input() always initially returns a string.

```
L164 [blank line]
```

Blank line used to separate logical sections and improve readability.

```
L165         try:
```

Starts an exception-handling block for an operation that may fail.

```
L166             number = int(value)
```

Converts the entered text to an integer; invalid text raises ValueError.

```
L167 [blank line]
```

Blank line used to separate logical sections and improve readability.

```
L168             if minimum <= number <= maximum:
```

Checks a condition and runs the indented block only when it is true.

```
L169                 return number
```

Returns the calculated value or selected record to the caller.

```
L170 [blank line]
```

Blank line used to separate logical sections and improve readability.

```
L171             print(f"Enter a whole number from {minimum} to {maximum}.")
```

Displays a message, result, warning, or formatted output to the terminal.

```
L172 [blank line]
```

Blank line used to separate logical sections and improve readability.

```
L173         except ValueError:
```

Handles this specific error so the program can respond gracefully instead of crashing.

```
L174             print("Invalid input! Enter a whole number only.")
```

Displays a message, result, warning, or formatted output to the terminal.

```
L175 [blank line]
```

Blank line used to separate logical sections and improve readability.

```
L176 [blank line]
```

Blank line used to separate logical sections and improve readability.

Function: get_number | Source lines around 177

```
L177 def get_number(prompt, minimum, maximum):
```

Defines a helper for numeric input within a range and rejects NaN/infinity.

```
L178     while True:
```

Repeats the block while the condition remains true.

```
L179         value = input(prompt).strip()
```

Reads text typed by the user; input() always initially returns a string.

```
L180 [blank line]
```

Blank line used to separate logical sections and improve readability.

```
L181         try:
```

Starts an exception-handling block for an operation that may fail.

```
L182             number = float(value)
```

Converts the entered text to a floating-point number.

```
L183 [blank line]
```

Blank line used to separate logical sections and improve readability.

```
L184             if not math.isfinite(number):
```

Checks a condition and runs the indented block only when it is true.

```
L185                 print("Invalid input! Enter a finite number.")
```

Displays a message, result, warning, or formatted output to the terminal.

```
L186                 continue
```

Skips the rest of this loop iteration and proceeds to the next item/input attempt.

```
L187 [blank line]
```

Blank line used to separate logical sections and improve readability.

```
L188             if minimum <= number <= maximum:
```

Checks a condition and runs the indented block only when it is true.

```
L189                 return number
```

Returns the calculated value or selected record to the caller.

```
L190 [blank line]
```

Blank line used to separate logical sections and improve readability.

```
L191             print(f"Enter a value from {minimum} to {maximum}.")
```

Displays a message, result, warning, or formatted output to the terminal.

```
L192 [blank line]
```

Blank line used to separate logical sections and improve readability.

```
L193         except ValueError:
```

Handles this specific error so the program can respond gracefully instead of crashing.

```
L194             print("Invalid input! Enter a valid number.")
```

Displays a message, result, warning, or formatted output to the terminal.

```
L195 [blank line]
```

Blank line used to separate logical sections and improve readability.

```
L196 [blank line]
```

Blank line used to separate logical sections and improve readability.

Function: get_yes_no | Source lines around 197

```
L197 def get_yes_no(prompt):
```

Defines a helper that accepts only y or n.

```
L198     while True:
```

Repeats the block while the condition remains true.

```
L199         answer = input(prompt).strip().lower()
```

Reads text typed by the user; input() always initially returns a string.

```
L200 [blank line]
```

Blank line used to separate logical sections and improve readability.

```
L201         if answer in ("y", "n"):
```

Checks a condition and runs the indented block only when it is true.

```
L202             return answer
```

Returns the calculated value or selected record to the caller.

```
L203 [blank line]
```

Blank line used to separate logical sections and improve readability.

```
L204         print("Invalid choice! Enter y or n.")
```

Displays a message, result, warning, or formatted output to the terminal.

```
L205 [blank line]
```

Blank line used to separate logical sections and improve readability.

```
L206 [blank line]
```

Blank line used to separate logical sections and improve readability.

```
L207 # ==================== STUDENT SEARCH ====================
```

Section heading comment: marks the beginning of a related group of functions; it does not execute.

```
L208 [blank line]
```

Blank line used to separate logical sections and improve readability.

Function: find_student | Source lines around 209

```
L209 def find_student(students, student_id):
```

Defines a case-insensitive search for a student by ID.

```
L210     for student in students:
```

Starts a loop that processes each item in the specified iterable.

```
L211         if student["id"].casefold() == student_id.casefold():
```

Checks a condition and runs the indented block only when it is true.

```
L212             return student
```

Returns the calculated value or selected record to the caller.

```
L213 [blank line]
```

Blank line used to separate logical sections and improve readability.

```
L214     return None
```

Returns None to indicate that no matching student was found.

```
L215 [blank line]
```

Blank line used to separate logical sections and improve readability.

```
L216 [blank line]
```

Blank line used to separate logical sections and improve readability.

```
L217 # ==================== GRADES AND RESULTS ====================
```

Section heading comment: marks the beginning of a related group of functions; it does not execute.

```
L218 [blank line]
```

Blank line used to separate logical sections and improve readability.

Function: get_grade | Source lines around 219

```
L219 def get_grade(marks):
```

Defines the marks-to-letter-grade conversion.

```
L220     if marks >= 90:
```

Checks a condition and runs the indented block only when it is true.

```
L221         return "A"
```

Returns the calculated value or selected record to the caller.

```
L222     elif marks >= 80:
```

Checks this alternative condition only if the preceding if/elif conditions were false.

```
L223         return "B"
```

Returns the calculated value or selected record to the caller.

```
L224     elif marks >= 70:
```

Checks this alternative condition only if the preceding if/elif conditions were false.

```
L225         return "C"
```

Returns the calculated value or selected record to the caller.

```
L226     elif marks >= 60:
```

Checks this alternative condition only if the preceding if/elif conditions were false.

```
L227         return "D"
```

Returns the calculated value or selected record to the caller.

```
L228     elif marks >= 35:
```

Checks this alternative condition only if the preceding if/elif conditions were false.

```
L229         return "E"
```

Returns the calculated value or selected record to the caller.

```
L230     else:
```

Runs this alternative block when preceding conditions are not satisfied.

```
L231         return "F"
```

Returns the calculated value or selected record to the caller.

```
L232 [blank line]
```

Blank line used to separate logical sections and improve readability.

```
L233 [blank line]
```

Blank line used to separate logical sections and improve readability.

Function: show_result | Source lines around 234

```
L234 def show_result(student):
```

Defines the function that prints pass/fail feedback and the student's grade.

```
L235     name = student["name"]
```

Reads the student's name field from the dictionary.

```
L236     course = student["course"]
```

Reads the student's course field from the dictionary.

```
L237     marks = student["marks"]
```

Reads the student's marks field from the dictionary.

```
L238 [blank line]
```

Blank line used to separate logical sections and improve readability.

```
L239     print("\n========== STUDENT RESULT ==========")
```

Displays a message, result, warning, or formatted output to the terminal.

```
L240 [blank line]
```

Blank line used to separate logical sections and improve readability.

```
L241     if marks < 35:
```

Checks a condition and runs the indented block only when it is true.

```
L242         print("Result: FAIL")
```

Displays a message, result, warning, or formatted output to the terminal.

```
L243         print(
```

Displays a message, result, warning, or formatted output to the terminal.

```
L244             f"You have failed in {course} "
```

Creates an f-string, inserting variable values into the displayed text.

```
L245             f"with {marks:g} marks, {name}."
```

Creates an f-string, inserting variable values into the displayed text.

```
L246         )
```

Closes the dictionary, call, list, or grouped expression opened above.

```
L247 [blank line]
```

Blank line used to separate logical sections and improve readability.

```
L248     elif marks >= 85:
```

Checks this alternative condition only if the preceding if/elif conditions were false.

```
L249         print("Result: PASS")
```

Displays a message, result, warning, or formatted output to the terminal.

```
L250         print(
```

Displays a message, result, warning, or formatted output to the terminal.

```
L251             f"Congratulations, {name}! You scored {marks:g} "
```

Creates an f-string, inserting variable values into the displayed text.

```
L252             f"marks in {course}. Excellent performance!"
```

Creates an f-string, inserting variable values into the displayed text.

```
L253         )
```

Closes the dictionary, call, list, or grouped expression opened above.

```
L254 [blank line]
```

Blank line used to separate logical sections and improve readability.

```
L255     else:
```

Runs this alternative block when preceding conditions are not satisfied.

```
L256         print("Result: PASS")
```

Displays a message, result, warning, or formatted output to the terminal.

```
L257         print(
```

Displays a message, result, warning, or formatted output to the terminal.

```
L258             f"Well done, {name}! You scored {marks:g} "
```

Creates an f-string, inserting variable values into the displayed text.

```
L259             f"marks in {course}."
```

Creates an f-string, inserting variable values into the displayed text.

```
L260         )
```

Closes the dictionary, call, list, or grouped expression opened above.

```
L261 [blank line]
```

Blank line used to separate logical sections and improve readability.

```
L262     print("Grade:", get_grade(marks))
```

Displays a message, result, warning, or formatted output to the terminal.

```
L263 [blank line]
```

Blank line used to separate logical sections and improve readability.

```
L264 [blank line]
```

Blank line used to separate logical sections and improve readability.

```
L265 # ==================== ADD STUDENT ====================
```

Section heading comment: marks the beginning of a related group of functions; it does not execute.

```
L266 [blank line]
```

Blank line used to separate logical sections and improve readability.

Function: add_student | Source lines around 267

```
L267 def add_student(students):
```

Defines the workflow for validating, adding, saving, and reporting a new student.

```
L268     student_id = get_student_id("Student ID: ")
```

Gets a validated student ID from the user.

```
L269 [blank line]
```

Blank line used to separate logical sections and improve readability.

```
L270     if find_student(students, student_id):
```

Checks a condition and runs the indented block only when it is true.

```
L271         print("Student ID already exists.")
```

Displays a message, result, warning, or formatted output to the terminal.

```
L272         return
```

Sends a value back to the code that called this function.

```
L273 [blank line]
```

Blank line used to separate logical sections and improve readability.

```
L274     name = get_student_name("Student name: ")
```

Collects a validated student name.

```
L275     age = get_integer("Age (1-120): ", 1, 120)
```

Collects an age as an integer in the allowed range.

```
L276     course = get_course("Course: ")
```

Collects a valid course name.

```
L277     marks = get_number("Marks (0-100): ", 0, 100)
```

Collects marks as a finite number from 0 through 100.

```
L278 [blank line]
```

Blank line used to separate logical sections and improve readability.

```
L279     student = {
```

Begins a dictionary representing one student's fields and values.

```
L280         "id": student_id,
```

Provides a string literal used as data, a message, or a dictionary key.

```
L281         "name": name,
```

Provides a string literal used as data, a message, or a dictionary key.

```
L282         "age": age,
```

Provides a string literal used as data, a message, or a dictionary key.

```
L283         "course": course,
```

Provides a string literal used as data, a message, or a dictionary key.

```
L284         "marks": marks
```

Provides a string literal used as data, a message, or a dictionary key.

```
L285     }
```

Closes the dictionary, call, list, or grouped expression opened above.

```
L286 [blank line]
```

Blank line used to separate logical sections and improve readability.

```
L287     students.append(student)
```

Adds the student dictionary to the in-memory list.

```
L288 [blank line]
```

Blank line used to separate logical sections and improve readability.

```
L289     if save_students(students):
```

Checks a condition and runs the indented block only when it is true.

```
L290         print("Student added successfully!")
```

Displays a message, result, warning, or formatted output to the terminal.

```
L291         show_result(student)
```

Part of the current expression: it supplies an argument, condition, value, or continuation to the statement above.

```
L292     else:
```

Runs this alternative block when preceding conditions are not satisfied.

```
L293         students.remove(student)
```

Removes the newly added record to roll back an unsuccessful save.

```
L294         print("Student was not added because saving failed.")
```

Displays a message, result, warning, or formatted output to the terminal.

```
L295 [blank line]
```

Blank line used to separate logical sections and improve readability.

```
L296 [blank line]
```

Blank line used to separate logical sections and improve readability.

```
L297 # ==================== VIEW STUDENTS ====================
```

Section heading comment: marks the beginning of a related group of functions; it does not execute.

```
L298 [blank line]
```

Blank line used to separate logical sections and improve readability.

Function: view_students | Source lines around 299

```
L299 def view_students(students):
```

Defines the formatted table display for all students.

```
L300     if not students:
```

Checks a condition and runs the indented block only when it is true.

```
L301         print("No student records found.")
```

Displays a message, result, warning, or formatted output to the terminal.

```
L302         return
```

Sends a value back to the code that called this function.

```
L303 [blank line]
```

Blank line used to separate logical sections and improve readability.

```
L304     print(
```

Displays a message, result, warning, or formatted output to the terminal.

```
L305         f"\n{'ID':<12}{'Name':<22}{'Age':<6}"
```

Creates an f-string, inserting variable values into the displayed text.

```
L306         f"{'Course':<22}{'Marks':<10}{'Grade'}"
```

Creates an f-string, inserting variable values into the displayed text.

```
L307     )
```

Closes the dictionary, call, list, or grouped expression opened above.

```
L308     print("-" * 80)
```

Displays a message, result, warning, or formatted output to the terminal.

```
L309 [blank line]
```

Blank line used to separate logical sections and improve readability.

```
L310     for student in students:
```

Starts a loop that processes each item in the specified iterable.

```
L311         print(
```

Displays a message, result, warning, or formatted output to the terminal.

```
L312             f"{student['id']:<12}"
```

Creates an f-string, inserting variable values into the displayed text.

```
L313             f"{student['name']:<22}"
```

Creates an f-string, inserting variable values into the displayed text.

```
L314             f"{student['age']:<6}"
```

Creates an f-string, inserting variable values into the displayed text.

```
L315             f"{student['course']:<22}"
```

Creates an f-string, inserting variable values into the displayed text.

```
L316             f"{student['marks']:<10.1f}"
```

Creates an f-string, inserting variable values into the displayed text.

```
L317             f"{get_grade(student['marks'])}"
```

Creates an f-string, inserting variable values into the displayed text.

```
L318         )
```

Closes the dictionary, call, list, or grouped expression opened above.

```
L319 [blank line]
```

Blank line used to separate logical sections and improve readability.

```
L320 [blank line]
```

Blank line used to separate logical sections and improve readability.

```
L321 # ==================== SEARCH STUDENT ====================
```

Section heading comment: marks the beginning of a related group of functions; it does not execute.

```
L322 [blank line]
```

Blank line used to separate logical sections and improve readability.

Function: search_student | Source lines around 323

```
L323 def search_student(students):
```

Defines the workflow for finding and displaying one student's details.

```
L324     student_id = get_student_id("Enter student ID: ")
```

Gets a validated student ID from the user.

```
L325     student = find_student(students, student_id)
```

Assigns the expression on the right to the variable or dictionary field on the left.

```
L326 [blank line]
```

Blank line used to separate logical sections and improve readability.

```
L327     if student:
```

Checks a condition and runs the indented block only when it is true.

```
L328         print("\nStudent details:")
```

Displays a message, result, warning, or formatted output to the terminal.

```
L329         print("ID:", student["id"])
```

Displays a message, result, warning, or formatted output to the terminal.

```
L330         print("Name:", student["name"])
```

Displays a message, result, warning, or formatted output to the terminal.

```
L331         print("Age:", student["age"])
```

Displays a message, result, warning, or formatted output to the terminal.

```
L332         print("Course:", student["course"])
```

Displays a message, result, warning, or formatted output to the terminal.

```
L333         print("Marks:", student["marks"])
```

Displays a message, result, warning, or formatted output to the terminal.

```
L334 [blank line]
```

Blank line used to separate logical sections and improve readability.

```
L335         show_result(student)
```

Part of the current expression: it supplies an argument, condition, value, or continuation to the statement above.

```
L336     else:
```

Runs this alternative block when preceding conditions are not satisfied.

```
L337         print("Student not found.")
```

Displays a message, result, warning, or formatted output to the terminal.

```
L338 [blank line]
```

Blank line used to separate logical sections and improve readability.

```
L339 [blank line]
```

Blank line used to separate logical sections and improve readability.

```
L340 # ==================== UPDATE STUDENT ====================
```

Section heading comment: marks the beginning of a related group of functions; it does not execute.

```
L341 [blank line]
```

Blank line used to separate logical sections and improve readability.

Function: update_student | Source lines around 342

```
L342 def update_student(students):
```

Defines the workflow for editing an existing student's information.

```
L343     student_id = get_student_id("Enter student ID to update: ")
```

Gets a validated student ID from the user.

```
L344     student = find_student(students, student_id)
```

Assigns the expression on the right to the variable or dictionary field on the left.

```
L345 [blank line]
```

Blank line used to separate logical sections and improve readability.

```
L346     if not student:
```

Checks a condition and runs the indented block only when it is true.

```
L347         print("Student not found.")
```

Displays a message, result, warning, or formatted output to the terminal.

```
L348         return
```

Sends a value back to the code that called this function.

```
L349 [blank line]
```

Blank line used to separate logical sections and improve readability.

```
L350     original = student.copy()
```

Makes a shallow copy of the current record so changes can be rolled back.

```
L351 [blank line]
```

Blank line used to separate logical sections and improve readability.

```
L352     print("Press Enter to keep the current value.")
```

Displays a message, result, warning, or formatted output to the terminal.

```
L353 [blank line]
```

Blank line used to separate logical sections and improve readability.

```
L354     name = input(f"Name [{student['name']}]: ").strip()
```

Reads text typed by the user; input() always initially returns a string.

```
L355 [blank line]
```

Blank line used to separate logical sections and improve readability.

```
L356     if name:
```

Checks a condition and runs the indented block only when it is true.

```
L357         while not is_valid_name(name):
```

Repeats the block while the condition remains true.

```
L358             print("Invalid name! Please use valid name characters.")
```

Displays a message, result, warning, or formatted output to the terminal.

```
L359             name = input("Enter name again (or Enter to skip): ").strip()
```

Reads text typed by the user; input() always initially returns a string.

```
L360 [blank line]
```

Blank line used to separate logical sections and improve readability.

```
L361             if not name:
```

Checks a condition and runs the indented block only when it is true.

```
L362                 break
```

Exits the current loop.

```
L363 [blank line]
```

Blank line used to separate logical sections and improve readability.

```
L364         if name:
```

Checks a condition and runs the indented block only when it is true.

```
L365             student["name"] = name
```

Updates a field in the selected student's dictionary.

```
L366 [blank line]
```

Blank line used to separate logical sections and improve readability.

```
L367     course = input(f"Course [{student['course']}]: ").strip()
```

Reads text typed by the user; input() always initially returns a string.

```
L368 [blank line]
```

Blank line used to separate logical sections and improve readability.

```
L369     if course:
```

Checks a condition and runs the indented block only when it is true.

```
L370         while not any(char.isalpha() for char in course):
```

Repeats the block while the condition remains true.

```
L371             print("Invalid course! Enter a course containing letters.")
```

Displays a message, result, warning, or formatted output to the terminal.

```
L372             course = input("Enter course again (or Enter to skip): ").strip()
```

Reads text typed by the user; input() always initially returns a string.

```
L373 [blank line]
```

Blank line used to separate logical sections and improve readability.

```
L374             if not course:
```

Checks a condition and runs the indented block only when it is true.

```
L375                 break
```

Exits the current loop.

```
L376 [blank line]
```

Blank line used to separate logical sections and improve readability.

```
L377         if course:
```

Checks a condition and runs the indented block only when it is true.

```
L378             student["course"] = course
```

Updates a field in the selected student's dictionary.

```
L379 [blank line]
```

Blank line used to separate logical sections and improve readability.

```
L380     if get_yes_no("Update age? (y/n): ") == "y":
```

Checks a condition and runs the indented block only when it is true.

```
L381         student["age"] = get_integer("New age (1-120): ", 1, 120)
```

Updates a field in the selected student's dictionary.

```
L382 [blank line]
```

Blank line used to separate logical sections and improve readability.

```
L383     if get_yes_no("Update marks? (y/n): ") == "y":
```

Checks a condition and runs the indented block only when it is true.

```
L384         student["marks"] = get_number(
```

Updates a field in the selected student's dictionary.

```
L385             "New marks (0-100): ", 0, 100
```

Provides a string literal used as data, a message, or a dictionary key.

```
L386         )
```

Closes the dictionary, call, list, or grouped expression opened above.

```
L387 [blank line]
```

Blank line used to separate logical sections and improve readability.

```
L388     if save_students(students):
```

Checks a condition and runs the indented block only when it is true.

```
L389         print("Student updated successfully!")
```

Displays a message, result, warning, or formatted output to the terminal.

```
L390         show_result(student)
```

Part of the current expression: it supplies an argument, condition, value, or continuation to the statement above.

```
L391     else:
```

Runs this alternative block when preceding conditions are not satisfied.

```
L392         student.update(original)
```

Restores the original dictionary values if saving an update fails.

```
L393         print("Update cancelled because saving failed.")
```

Displays a message, result, warning, or formatted output to the terminal.

```
L394 [blank line]
```

Blank line used to separate logical sections and improve readability.

```
L395 [blank line]
```

Blank line used to separate logical sections and improve readability.

```
L396 # ==================== DELETE STUDENT ====================
```

Section heading comment: marks the beginning of a related group of functions; it does not execute.

```
L397 [blank line]
```

Blank line used to separate logical sections and improve readability.

Function: delete_student | Source lines around 398

```
L398 def delete_student(students):
```

Defines the workflow for confirming and deleting a student record.

```
L399     student_id = get_student_id("Enter student ID to delete: ")
```

Gets a validated student ID from the user.

```
L400     student = find_student(students, student_id)
```

Assigns the expression on the right to the variable or dictionary field on the left.

```
L401 [blank line]
```

Blank line used to separate logical sections and improve readability.

```
L402     if not student:
```

Checks a condition and runs the indented block only when it is true.

```
L403         print("Student not found.")
```

Displays a message, result, warning, or formatted output to the terminal.

```
L404         return
```

Sends a value back to the code that called this function.

```
L405 [blank line]
```

Blank line used to separate logical sections and improve readability.

```
L406     if get_yes_no(
```

Checks a condition and runs the indented block only when it is true.

```
L407         f"Delete {student['name']}? (y/n): "
```

Creates an f-string, inserting variable values into the displayed text.

```
L408     ) == "y":
```

Begins an indented block that belongs to this function, loop, condition, or context manager.

```
L409         index = students.index(student)
```

Stores the record's list position so it can be restored if needed.

```
L410         students.pop(index)
```

Removes the student at the selected index.

```
L411 [blank line]
```

Blank line used to separate logical sections and improve readability.

```
L412         if save_students(students):
```

Checks a condition and runs the indented block only when it is true.

```
L413             print("Student deleted successfully!")
```

Displays a message, result, warning, or formatted output to the terminal.

```
L414         else:
```

Runs this alternative block when preceding conditions are not satisfied.

```
L415             students.insert(index, student)
```

Restores the deleted student at its original position if saving fails.

```
L416             print("Deletion cancelled because saving failed.")
```

Displays a message, result, warning, or formatted output to the terminal.

```
L417     else:
```

Runs this alternative block when preceding conditions are not satisfied.

```
L418         print("Deletion cancelled.")
```

Displays a message, result, warning, or formatted output to the terminal.

```
L419 [blank line]
```

Blank line used to separate logical sections and improve readability.

```
L420 [blank line]
```

Blank line used to separate logical sections and improve readability.

```
L421 # ==================== STATISTICS ====================
```

Section heading comment: marks the beginning of a related group of functions; it does not execute.

```
L422 [blank line]
```

Blank line used to separate logical sections and improve readability.

Function: statistics | Source lines around 423

```
L423 def statistics(students):
```

Defines summary calculations such as average, topper, and number of failures.

```
L424     if not students:
```

Checks a condition and runs the indented block only when it is true.

```
L425         print("No records available.")
```

Displays a message, result, warning, or formatted output to the terminal.

```
L426         return
```

Sends a value back to the code that called this function.

```
L427 [blank line]
```

Blank line used to separate logical sections and improve readability.

```
L428     average = sum(s["marks"] for s in students) / len(students)
```

Calculates the arithmetic mean of all students' marks.

```
L429     topper = max(students, key=lambda s: s["marks"])
```

Finds the student dictionary with the highest marks.

```
L430     failures = sum(1 for s in students if s["marks"] < 35)
```

Counts records whose marks are below the pass threshold of 35.

```
L431 [blank line]
```

Blank line used to separate logical sections and improve readability.

```
L432     print("\n========== STUDENT STATISTICS ==========")
```

Displays a message, result, warning, or formatted output to the terminal.

```
L433     print("Total students:", len(students))
```

Displays a message, result, warning, or formatted output to the terminal.

```
L434     print(f"Average marks: {average:.2f}")
```

Displays a message, result, warning, or formatted output to the terminal.

```
L435     print("Topper:", topper["name"])
```

Displays a message, result, warning, or formatted output to the terminal.

```
L436     print("Highest marks:", topper["marks"])
```

Displays a message, result, warning, or formatted output to the terminal.

```
L437     print("Topper's grade:", get_grade(topper["marks"]))
```

Displays a message, result, warning, or formatted output to the terminal.

```
L438     print("Students who failed:", failures)
```

Displays a message, result, warning, or formatted output to the terminal.

```
L439 [blank line]
```

Blank line used to separate logical sections and improve readability.

```
L440 [blank line]
```

Blank line used to separate logical sections and improve readability.

```
L441 # ==================== SORT STUDENTS ====================
```

Section heading comment: marks the beginning of a related group of functions; it does not execute.

```
L442 [blank line]
```

Blank line used to separate logical sections and improve readability.

Function: sort_students | Source lines around 443

```
L443 def sort_students(students):
```

Defines the menu for displaying records sorted by name or marks.

```
L444     if not students:
```

Checks a condition and runs the indented block only when it is true.

```
L445         print("No student records found.")
```

Displays a message, result, warning, or formatted output to the terminal.

```
L446         return
```

Sends a value back to the code that called this function.

```
L447 [blank line]
```

Blank line used to separate logical sections and improve readability.

```
L448     print("1. Sort by name")
```

Displays a message, result, warning, or formatted output to the terminal.

```
L449     print("2. Sort by marks (highest first)")
```

Displays a message, result, warning, or formatted output to the terminal.

```
L450 [blank line]
```

Blank line used to separate logical sections and improve readability.

```
L451     choice = input("Choose (1-2): ").strip()
```

Reads text typed by the user; input() always initially returns a string.

```
L452 [blank line]
```

Blank line used to separate logical sections and improve readability.

```
L453     if choice == "1":
```

Checks a condition and runs the indented block only when it is true.

```
L454         ordered = sorted(
```

Stores a new sorted list; the original student list is not reordered.

```
L455             students,
```

Part of the current expression: it supplies an argument, condition, value, or continuation to the statement above.

```
L456             key=lambda s: s["name"].casefold()
```

Specifies the value used to compare records during sorting or selection.

```
L457         )
```

Closes the dictionary, call, list, or grouped expression opened above.

```
L458     elif choice == "2":
```

Checks this alternative condition only if the preceding if/elif conditions were false.

```
L459         ordered = sorted(
```

Stores a new sorted list; the original student list is not reordered.

```
L460             students,
```

Part of the current expression: it supplies an argument, condition, value, or continuation to the statement above.

```
L461             key=lambda s: s["marks"],
```

Specifies the value used to compare records during sorting or selection.

```
L462             reverse=True
```

Controls sort direction; True means descending order.

```
L463         )
```

Closes the dictionary, call, list, or grouped expression opened above.

```
L464     else:
```

Runs this alternative block when preceding conditions are not satisfied.

```
L465         print("Invalid choice! Enter 1 or 2.")
```

Displays a message, result, warning, or formatted output to the terminal.

```
L466         return
```

Sends a value back to the code that called this function.

```
L467 [blank line]
```

Blank line used to separate logical sections and improve readability.

```
L468     view_students(ordered)
```

Part of the current expression: it supplies an argument, condition, value, or continuation to the statement above.

```
L469 [blank line]
```

Blank line used to separate logical sections and improve readability.

```
L470 [blank line]
```

Blank line used to separate logical sections and improve readability.

```
L471 # ==================== AUTOMATIC TESTS ====================
```

Section heading comment: marks the beginning of a related group of functions; it does not execute.

```
L472 [blank line]
```

Blank line used to separate logical sections and improve readability.

Function: run_tests | Source lines around 473

```
L473 def run_tests():
```

Defines automated assertions that check validation, grades, and result messages.

```
L474     print("\n========== RUNNING TESTS ==========")
```

Displays a message, result, warning, or formatted output to the terminal.

```
L475 [blank line]
```

Blank line used to separate logical sections and improve readability.

```
L476     # Test ID validation rules
```

Comment for the reader; Python ignores this line during execution.

```
L477     valid_ids = ["ST101", "KLU2026"]
```

Assigns the expression on the right to the variable or dictionary field on the left.

```
L478     invalid_ids = ["ST@101", "ST 101", "ST#101", ""]
```

Assigns the expression on the right to the variable or dictionary field on the left.

```
L479 [blank line]
```

Blank line used to separate logical sections and improve readability.

```
L480     for student_id in valid_ids:
```

Starts a loop that processes each item in the specified iterable.

```
L481         assert (
```

Checks a test condition; if false, Python raises AssertionError.

```
L482             student_id.isascii() and student_id.isalnum()
```

Part of the current expression: it supplies an argument, condition, value, or continuation to the statement above.

```
L483         )
```

Closes the dictionary, call, list, or grouped expression opened above.

```
L484         print(f"PASS: Valid ID - {student_id}")
```

Displays a message, result, warning, or formatted output to the terminal.

```
L485 [blank line]
```

Blank line used to separate logical sections and improve readability.

```
L486     for student_id in invalid_ids:
```

Starts a loop that processes each item in the specified iterable.

```
L487         assert not (
```

Checks a test condition; if false, Python raises AssertionError.

```
L488             student_id
```

Part of the current expression: it supplies an argument, condition, value, or continuation to the statement above.

```
L489             and student_id.isascii()
```

Combines or negates Boolean conditions used for validation.

```
L490             and student_id.isalnum()
```

Combines or negates Boolean conditions used for validation.

```
L491         )
```

Closes the dictionary, call, list, or grouped expression opened above.

```
L492         print(f"PASS: Invalid ID rejected - {student_id!r}")
```

Displays a message, result, warning, or formatted output to the terminal.

```
L493 [blank line]
```

Blank line used to separate logical sections and improve readability.

```
L494     # Test name validation
```

Comment for the reader; Python ignores this line during execution.

```
L495     assert is_valid_name("Charan Sai")
```

Checks a test condition; if false, Python raises AssertionError.

```
L496     assert is_valid_name("Anne-Marie")
```

Checks a test condition; if false, Python raises AssertionError.

```
L497     assert not is_valid_name("Charan123")
```

Checks a test condition; if false, Python raises AssertionError.

```
L498     assert not is_valid_name("12345")
```

Checks a test condition; if false, Python raises AssertionError.

```
L499     print("PASS: Student name validation")
```

Displays a message, result, warning, or formatted output to the terminal.

```
L500 [blank line]
```

Blank line used to separate logical sections and improve readability.

```
L501     # Test grades and result boundaries
```

Comment for the reader; Python ignores this line during execution.

```
L502     grade_tests = {
```

Assigns the expression on the right to the variable or dictionary field on the left.

```
L503         0: "F",
```

Part of the current expression: it supplies an argument, condition, value, or continuation to the statement above.

```
L504         34: "F",
```

Part of the current expression: it supplies an argument, condition, value, or continuation to the statement above.

```
L505         35: "E",
```

Part of the current expression: it supplies an argument, condition, value, or continuation to the statement above.

```
L506         59: "E",
```

Part of the current expression: it supplies an argument, condition, value, or continuation to the statement above.

```
L507         60: "D",
```

Part of the current expression: it supplies an argument, condition, value, or continuation to the statement above.

```
L508         70: "C",
```

Part of the current expression: it supplies an argument, condition, value, or continuation to the statement above.

```
L509         80: "B",
```

Part of the current expression: it supplies an argument, condition, value, or continuation to the statement above.

```
L510         84: "B",
```

Part of the current expression: it supplies an argument, condition, value, or continuation to the statement above.

```
L511         85: "B",
```

Part of the current expression: it supplies an argument, condition, value, or continuation to the statement above.

```
L512         90: "A",
```

Part of the current expression: it supplies an argument, condition, value, or continuation to the statement above.

```
L513         100: "A"
```

Part of the current expression: it supplies an argument, condition, value, or continuation to the statement above.

```
L514     }
```

Closes the dictionary, call, list, or grouped expression opened above.

```
L515 [blank line]
```

Blank line used to separate logical sections and improve readability.

```
L516     for marks, expected in grade_tests.items():
```

Starts a loop that processes each item in the specified iterable.

```
L517         assert get_grade(marks) == expected
```

Checks a test condition; if false, Python raises AssertionError.

```
L518         print(f"PASS: {marks} marks -> Grade {expected}")
```

Displays a message, result, warning, or formatted output to the terminal.

```
L519 [blank line]
```

Blank line used to separate logical sections and improve readability.

```
L520     # Test student feedback without requesting keyboard input
```

Comment for the reader; Python ignores this line during execution.

```
L521     import contextlib
```

Imports the contextlib module so its functions can be used.

```
L522     import io
```

Imports the io module so its functions can be used.

```
L523 [blank line]
```

Blank line used to separate logical sections and improve readability.

```
L524     sample = {
```

Assigns the expression on the right to the variable or dictionary field on the left.

```
L525         "id": "ST101",
```

Provides a string literal used as data, a message, or a dictionary key.

```
L526         "name": "Ravi",
```

Provides a string literal used as data, a message, or a dictionary key.

```
L527         "age": 20,
```

Provides a string literal used as data, a message, or a dictionary key.

```
L528         "course": "Computer Science",
```

Provides a string literal used as data, a message, or a dictionary key.

```
L529         "marks": 34
```

Provides a string literal used as data, a message, or a dictionary key.

```
L530     }
```

Closes the dictionary, call, list, or grouped expression opened above.

```
L531 [blank line]
```

Blank line used to separate logical sections and improve readability.

```
L532     output = io.StringIO()
```

Creates an in-memory text buffer to capture printed output during a test.

```
L533 [blank line]
```

Blank line used to separate logical sections and improve readability.

```
L534     with contextlib.redirect_stdout(output):
```

Temporarily redirects print output into the test buffer instead of the terminal.

```
L535         show_result(sample)
```

Part of the current expression: it supplies an argument, condition, value, or continuation to the statement above.

```
L536 [blank line]
```

Blank line used to separate logical sections and improve readability.

```
L537     assert "FAIL" in output.getvalue()
```

Checks a test condition; if false, Python raises AssertionError.

```
L538     assert "Computer Science" in output.getvalue()
```

Checks a test condition; if false, Python raises AssertionError.

```
L539     assert "34" in output.getvalue()
```

Checks a test condition; if false, Python raises AssertionError.

```
L540 [blank line]
```

Blank line used to separate logical sections and improve readability.

```
L541     sample["marks"] = 85
```

Changes the sample record's marks to test another result scenario.

```
L542     output = io.StringIO()
```

Creates an in-memory text buffer to capture printed output during a test.

```
L543 [blank line]
```

Blank line used to separate logical sections and improve readability.

```
L544     with contextlib.redirect_stdout(output):
```

Temporarily redirects print output into the test buffer instead of the terminal.

```
L545         show_result(sample)
```

Part of the current expression: it supplies an argument, condition, value, or continuation to the statement above.

```
L546 [blank line]
```

Blank line used to separate logical sections and improve readability.

```
L547     assert "Congratulations, Ravi!" in output.getvalue()
```

Checks a test condition; if false, Python raises AssertionError.

```
L548     assert "85" in output.getvalue()
```

Checks a test condition; if false, Python raises AssertionError.

```
L549 [blank line]
```

Blank line used to separate logical sections and improve readability.

```
L550     print("PASS: Fail message")
```

Displays a message, result, warning, or formatted output to the terminal.

```
L551     print("PASS: Congratulations message")
```

Displays a message, result, warning, or formatted output to the terminal.

```
L552     print("\nAll automatic tests passed!")
```

Displays a message, result, warning, or formatted output to the terminal.

```
L553 [blank line]
```

Blank line used to separate logical sections and improve readability.

```
L554 [blank line]
```

Blank line used to separate logical sections and improve readability.

```
L555 # ==================== MAIN MENU ====================
```

Section heading comment: marks the beginning of a related group of functions; it does not execute.

```
L556 [blank line]
```

Blank line used to separate logical sections and improve readability.

Function: main | Source lines around 557

```
L557 def main():
```

Defines the main program loop and menu.

```
L558     students = load_students()
```

Assigns the expression on the right to the variable or dictionary field on the left.

```
L559 [blank line]
```

Blank line used to separate logical sections and improve readability.

```
L560     while True:
```

Repeats the block while the condition remains true.

```
L561         print("\n===== STUDENT MANAGEMENT SYSTEM =====")
```

Displays a message, result, warning, or formatted output to the terminal.

```
L562         print("1. Add Student")
```

Displays a message, result, warning, or formatted output to the terminal.

```
L563         print("2. View Students")
```

Displays a message, result, warning, or formatted output to the terminal.

```
L564         print("3. Search Student")
```

Displays a message, result, warning, or formatted output to the terminal.

```
L565         print("4. Update Student")
```

Displays a message, result, warning, or formatted output to the terminal.

```
L566         print("5. Delete Student")
```

Displays a message, result, warning, or formatted output to the terminal.

```
L567         print("6. Student Statistics")
```

Displays a message, result, warning, or formatted output to the terminal.

```
L568         print("7. Sort Students")
```

Displays a message, result, warning, or formatted output to the terminal.

```
L569         print("8. Run Automatic Tests")
```

Displays a message, result, warning, or formatted output to the terminal.

```
L570         print("9. Exit")
```

Displays a message, result, warning, or formatted output to the terminal.

```
L571 [blank line]
```

Blank line used to separate logical sections and improve readability.

```
L572         choice = input("Enter your choice (1-9): ").strip()
```

Reads text typed by the user; input() always initially returns a string.

```
L573 [blank line]
```

Blank line used to separate logical sections and improve readability.

```
L574         if choice == "1":
```

Checks a condition and runs the indented block only when it is true.

```
L575             add_student(students)
```

Part of the current expression: it supplies an argument, condition, value, or continuation to the statement above.

```
L576         elif choice == "2":
```

Checks this alternative condition only if the preceding if/elif conditions were false.

```
L577             view_students(students)
```

Part of the current expression: it supplies an argument, condition, value, or continuation to the statement above.

```
L578         elif choice == "3":
```

Checks this alternative condition only if the preceding if/elif conditions were false.

```
L579             search_student(students)
```

Part of the current expression: it supplies an argument, condition, value, or continuation to the statement above.

```
L580         elif choice == "4":
```

Checks this alternative condition only if the preceding if/elif conditions were false.

```
L581             update_student(students)
```

Part of the current expression: it supplies an argument, condition, value, or continuation to the statement above.

```
L582         elif choice == "5":
```

Checks this alternative condition only if the preceding if/elif conditions were false.

```
L583             delete_student(students)
```

Part of the current expression: it supplies an argument, condition, value, or continuation to the statement above.

```
L584         elif choice == "6":
```

Checks this alternative condition only if the preceding if/elif conditions were false.

```
L585             statistics(students)
```

Part of the current expression: it supplies an argument, condition, value, or continuation to the statement above.

```
L586         elif choice == "7":
```

Checks this alternative condition only if the preceding if/elif conditions were false.

```
L587             sort_students(students)
```

Part of the current expression: it supplies an argument, condition, value, or continuation to the statement above.

```
L588         elif choice == "8":
```

Checks this alternative condition only if the preceding if/elif conditions were false.

```
L589             run_tests()
```

Part of the current expression: it supplies an argument, condition, value, or continuation to the statement above.

```
L590         elif choice == "9":
```

Checks this alternative condition only if the preceding if/elif conditions were false.

```
L591             if save_students(students):
```

Checks a condition and runs the indented block only when it is true.

```
L592                 print("Records saved. Goodbye!")
```

Displays a message, result, warning, or formatted output to the terminal.

```
L593             else:
```

Runs this alternative block when preceding conditions are not satisfied.

```
L594                 print("Warning: records could not be saved.")
```

Displays a message, result, warning, or formatted output to the terminal.

```
L595             break
```

Exits the current loop.

```
L596         else:
```

Runs this alternative block when preceding conditions are not satisfied.

```
L597             print("Invalid choice! Enter a number from 1 to 9.")
```

Displays a message, result, warning, or formatted output to the terminal.

```
L598 [blank line]
```

Blank line used to separate logical sections and improve readability.

```
L599 [blank line]
```

Blank line used to separate logical sections and improve readability.

```
L600 if __name__ == "__main__":
```

Checks whether this file is being run directly; imported use will not automatically start the menu.

```
L601     main()
```

Calls main() to start the Student Management System.

3. Explanation of the major components

File handling — load_students()

Opens the JSON file and parses it. It checks that the top-level value is a list, each record is a dictionary, required keys exist, text fields have suitable types, IDs and names follow validation rules, age is an integer from 1 to 120, and marks are finite numbers from 0 to 100. Invalid records are skipped. Missing or malformed files are handled without crashing.

File handling — save_students()

Writes the current list to JSON using readable indentation and UTF-8. allow_nan=False prevents invalid JSON numeric values. It returns True on success and False if writing fails.

Input validation

Input helpers repeat until the user supplies valid data. strip() removes surrounding whitespace. isascii() and isalnum() restrict IDs to ASCII letters and digits. Names may contain letters, spaces, hyphens, apostrophes, and periods, but must include at least one letter. Marks accept decimal numbers from 0 to 100.

Searching

find_student() compares IDs using casefold(), so ST101 and st101 match. It returns the dictionary when found, or None when no match exists.

Grades and result messages

get_grade() converts marks to a letter grade. show_result() prints FAIL below 35, PASS otherwise, and adds an extra congratulatory message for marks of at least 85.

CRUD operations

Add creates a dictionary and appends it; view prints all records; search displays one record; update edits selected fields; delete asks for confirmation. Saving failures are handled by reverting the in-memory change.

Statistics and sorting

statistics() computes the count, mean marks, topper, highest score, topper grade, and number of students below 35. sort_students() uses sorted() to display records by name or marks without changing the original list order.

Automatic tests

run_tests() uses assert statements for ID validation, name validation, grade boundaries, and pass/fail messages. contextlib.redirect_stdout() and io.StringIO() capture printed text so it can be tested without requiring keyboard input.

Main menu

main() loads the saved records once, repeatedly prints the menu, dispatches the selected operation, and saves before exiting. The if __name__ == '__main__' guard starts the menu only when the file is executed directly.

4. Important Python syntax used

`import` — Makes a module's features available to this file.

`def function_name(...):` — Defines a reusable function; indented lines form its body.

`return` — Sends a result back to the caller and ends that function call.

`if / elif / else` — Chooses which code block to execute based on conditions.

`for item in collection` — Visits items one at a time.

`while True` — Repeats until a break or return exits the loop.

`try / except` — Runs code that may fail and handles specified errors.

`dictionary: {'id': value}` — Stores named fields for one student.

`list.append(value)` — Adds an item to a list.

`f'...{variable}...'` — Inserts values into a string; formats such as {marks:g} avoid unnecessary trailing zeros.

`assert condition` — Raises AssertionError if a test condition is false.

`lambda s: ...` — Creates a short anonymous function, used here as a sorting key.

`casefold()` — Normalizes text for robust case-insensitive comparison.

`math.isfinite(value)` — Rejects infinity and NaN values.

5. Final conclusion

The Student Management System is a practical core-Python project that combines input handling, conditional logic, loops, functions, dictionaries, lists, file operations, exception handling, data validation, sorting, basic statistics, and automated tests. Its JSON file allows records to remain available after the program closes. The program is divided into small functions, which makes it easier to read, test, debug, and extend.

Overall conclusion: this is a strong beginner-to-intermediate console project and a useful portfolio exercise. Before presenting it as a finished project, consider adding a few refinements: use a script-relative path if you want students.json always beside the Python file; remove or use get_non_empty_text(), which is currently defined but not called; test duplicate IDs in mixed-case form; and consider writing to a temporary file before replacing the JSON file to reduce the risk of losing data if writing is interrupted.

Recommended run steps

• Save the Python code as student_management.py.

• Open a terminal in the folder where you want students.json to be created.

• Run: python student_management.py

• Choose option 8 to run automatic tests.

• Choose option 1 to add a student, then options 2–7 to explore the other features.

• Choose option 9 to save and exit.

End of guide